

Dashboard Snapshot — Function Design (機能設計)

003_DD-FN — used by 003_DD and 003_DD-API, requirements REQ-028–REQ-042.

Document control (改版履歴)

Field	Value
Document ID	003_DD-FN
System name	ProductionManagementAI
Subsystem name	Production orders
Work item	WI-004
Created by	Claude (for ThanhTN)
Created date	2026-09-22
Last updated by	Claude (for ThanhTN)
Last updated date	2026-09-22

Version	Date	Author	Revision content
1	2026-09-22	Claude (for ThanhTN)	Initial creation
2	2026-09-22	Claude (for ThanhTN)	§6 <code>SystemHealthService.CheckAsync</code> and <code>IDatabasePing</code> (DEC-017, DEC-020); §7 the no-renew rule for the health path (DEC-019); observability extended
3	2026-09-22	Claude (for ThanhTN)	Implementation alignment (WI-004 DEC-024): §3 runs its statements through ADO.NET commands on the EF connection/transaction instead of <code>SqlQuery<T></code> ; health code in the <code>Health</code> namespace
4	2026-09-22	Claude (for ThanhTN)	§7: renewal also suppressed after security-stamp revalidation (WI-004 DEC-025, found by TC-225)

Overview and method index

Field	Value
Module name	<code>DashboardService</code> and its reader port (<code>ProductionManagementAI.Application.Dashboard</code> , reader implemented in <code>ProductionManagementAI.Infrastructure.Dashboard</code>)
Overview	Turn "now" into the dashboard's figures: derive every window from the plant-local date, read 003_DB's seven statements in one snapshot, and shape the result. A separate service from <code>ProductionOrderService</code> , because it shares no use case with it — only the clock, the telemetry source and the result conventions, which it reuses

No	Method name	Overview	Notes
1	<code>DashboardService.GetSnapshotAsync</code>	Read the clock once, build the window, read, shape, record telemetry	API-DSH-01
2	<code>DashboardWindow.For</code>	Every date and instant the figures depend on, from T — a pure function	New
3	<code>IDashboardReader.ReadAsync</code>	003_DB Q1–Q6 in one <code>REPEATABLE READ READ ONLY</code> transaction	New port; <code>DashboardReader</code> in Infrastructure
4	<code>DashboardMapper.ToResponse</code>	Raw rows → <code>DashboardResponse</code> : zero-filling, bucket order, lead-time rounding	New, pure
5	<code>IPlantClock.DateOf / StartOfDayUtc / TimeZoneId</code>	Plant-local calendar conversions the window needs	Additive members on the existing port (001_DD-FN §6)
6	<code>SystemHealthService.CheckAsync</code>	Ping the database with a 2 s timeout	API-SYS-01; port <code>IDatabasePing</code> , <code>DatabasePing</code> in Infrastructure
7	Cookie <code>OnCheckSlidingExpiration</code>	Health requests never renew the session	<code>DependencyInjection.cs</code>

Shared utility references

No	Name	Overview	Notes
1	<code>IPlantClock</code>	Plant-local date and zone	Existing port; gains three members (§5). <code>Today</code> and <code>CurrentYear</code> are unchanged
2	<code>TimeProvider</code>	The single "now" read per snapshot	Existing; <code>FakeTimeProvider</code> in tests pins it
3	<code>ProductionOrderTelemetry</code>	<code>ActivitySource</code> + <code>Meter</code> named <code>ProductionManagementAI.ProductionOrders</code>	Existing; gains one counter (see Observability). Reusing it means <code>Program.cs</code> registers no new source or meter
4	<code>ILogger<DashboardService></code>	Structured logging, <code>LoggerMessage</code> source-generated	

No	Name	Overview	Notes
5	AppDbContext	EF Core context; the reader uses <code>Database.SqlQuery<T></code> (<code>FormattableString</code>)	Interpolated values become bound parameters, never concatenated SQL (<code>ai/rules/database.md</code>)

Request data

Parameters sent to the database — exactly 003_DB's parameter table, produced by `DashboardWindow` (§2). All are bound parameters.

No	Name	Variable name	Type	Length	Used by (which method(s))	Notes
1	Plant today	<code>Today</code>	<code>DateOnly</code>	—	Q2, Q3	T
2	Due-soon end	<code>SoonEnd</code>	<code>DateOnly</code>	—	Q3b	T + 7 (DEC-007)
3	Week 0	<code>Week0</code>	<code>DateOnly</code>	—	Q2	W(T), the Monday on or before T
4	Trend start	<code>TrendStartUtc</code>	<code>DateTimeOffset</code>	—	Q5, Q6	Start of plant day W(T) – 77
5	Week start	<code>WeekStartUtc</code>	<code>DateTimeOffset</code>	—	Q5	Start of plant day W(T)
6	Month start	<code>MonthStartUtc</code>	<code>DateTimeOffset</code>	—	Q5	Start of plant day 1 of T's month
7	Window start	<code>WindowStartUtc</code>	<code>DateTimeOffset</code>	—	Q5	Start of plant day T – 29
8	End	<code>EndUtc</code>	<code>DateTimeOffset</code>	—	Q5, Q6	Start of plant day T + 1
9	Zone	<code>TimeZoneId</code>	<code>string</code>	—	Q5, Q6	<code>PlantOptions.TimeZone</code> , validated at startup

Response / value mapping

`DashboardResponse` (field catalog in 003_DD-API) is built by `DashboardMapper.ToResponse` (§4) from `DashboardRaw` , the reader's result:

No	Name	Variable name	Type	Length	Required	Value mapping	Notes
1	Status counts	<code>StatusCounts</code>	<code>(string status, int count)[]</code>	≤ 4	yes	Missing statuses → 0; <code>Total</code> = sum	Q1
2	Workload rows	<code>Workload</code>	<code>(int bucket, int count, long quantity)[]</code>	≤ 10	yes	<code>bucket</code> –1 → <code>overdue</code> , 0–7 → <code>week k</code> , 8 → <code>later</code> ; missing buckets → 0	Q2
3	Overdue / due-soon rows	<code>Overdue</code> , <code>DueSoon</code>	<code>(DashboardOrderRow[] rows, int total)</code>	≤ 10 rows	yes	<code>total</code> from <code>count(*) OVER ()</code> of the first row, 0 when no row	Q3a, Q3b

No	Name	Variable name	Type	Length	Required	Value mapping	Notes
4	Top products	TopProducts	TopProductRow[]	≤ 10	yes	As returned, already ordered	Q4
5	Delivery row	Delivery	DeliveryRow	1	yes	NULL sums → 0; NULL average → null	Q5
6	Trend rows	Trend	(DateOnly weekStart, int count)[]	≤ 12	yes	Laid onto the 12 expected Mondays; missing → 0	Q6

Method design

1. DashboardService.GetSnapshotAsync

Field	Value
Description	Build the complete dashboard snapshot for the current moment
Return type	Task<DashboardResponse>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	CancellationToken	cancellationToken	

Return value

Type	Name	Description
DashboardResponse	snapshot	Always complete. The method has no business failure — no input to reject, nothing to find — so it returns the response directly rather than a Result<T> ; unexpected exceptions propagate to the global Problem Details handler (500, MSG-E013), as in 002_DD-FN

Processing overview: the clock is read **once**, and T, every window, the `asOf` time and the response's `today` all derive from that one reading. A dashboard computed across midnight therefore cannot mix two "todays".

Processing flow

Step	Description	Calls
1	Start span <code>ProductionOrder.Dashboard</code>	<code>ProductionOrderTelemetry.Source</code>
2	<code>utcNow = timeProvider.GetUtcNow(); today = plantClock.DateOf(utcNow)</code>	<code>TimeProvider</code> , <code>IPlantClock</code> (§5)
3	<code>window = DashboardWindow.For(today, plantClock)</code>	§2
4	<code>raw = await reader.ReadAsync(window, ct)</code>	§3
5	<code>response = DashboardMapper.ToResponse(window, raw, utcNow, plantClock.TimeZoneId)</code>	§4
6	Tag the span (<code>dashboard.today</code> , <code>dashboard.active_orders</code> = draft + inProgress, <code>dashboard.window_completed</code> = onTime.completedCount); counter <code>dashboard_loaded{outcome=success}</code> ; return	telemetry
7	On exception: counter <code>dashboard_loaded{outcome=error}</code> , span status <code>Error</code> , log <code>DashboardSnapshotFailed</code> with the exception, rethrow	telemetry, logger

2. `DashboardWindow.For`

Field	Value
Description	Compute every date and UTC instant that 003_BD D-01–D-09 and 003_DB Q1–Q6 depend on, from the plant-local date T
Return type	<code>DashboardWindow</code> (immutable record)
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	<code>DateOnly</code>	<code>today</code>	T
2	<code>IPlantClock</code>	<code>clock</code>	For <code>StartOfDayUtc</code> and <code>TimeZoneId</code> only

Return value

Type	Name	Description
<code>DashboardWindow</code>	<code>window</code>	The nine fields in Request data, plus <code>MonthStart</code> (date), <code>WindowStart</code> (T – 29) and <code>TrendWeeks</code> , the 12 Mondays W(T) – 77 ... W(T)

Processing overview: a pure function of T and the zone. Every calendar rule lives here once, so it can be unit-tested for every weekday and month boundary without a database (TC-205, TC-209).

Processing flow

Step	Description	Calls
1	<code>Week0 = today.AddDays(-((7 + (int)today.DayOfWeek - 1) % 7))</code> — the Monday on or before T (Sunday maps to 6 days back)	—
2	<code>SoonEnd = today + 7 ; WindowStart = today - 29 ; MonthStart = new DateOnly(today.Year, today.Month, 1)</code>	—
3	<code>TrendWeeks = [Week0 - 77, Week0 - 70, ..., Week0]</code> (12 Mondays)	—
4	Convert each date lower bound to the start of that plant day in UTC: <code>TrendStartUtc = StartOfDayUtc(TrendWeeks[0])</code> , <code>WeekStartUtc = StartOfDayUtc(Week0)</code> , <code>MonthStartUtc</code> , <code>WindowStartUtc</code> , <code>EndUtc = StartOfDayUtc(today + 1)</code>	<code>IPlantClock.StartOfDayUtc</code>
5	<code>TimeZoneId = clock.TimeZoneId</code>	—

3. `IDashboardReader.ReadAsync` (`DashboardReader`)

Field	Value
Description	Run 003_DB Q1–Q6 (seven statements) inside one read-only snapshot transaction
Return type	<code>Task<DashboardRaw></code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	<code>DashboardWindow</code>	<code>window</code>	Parameters
2	<code>CancellationToken</code>	<code>cancellationToken</code>	

Return value

Type	Name	Description
<code>DashboardRaw</code>	<code>raw</code>	The seven results, unshaped

Processing overview: raw SQL, because Q2's bucket expression, Q3's window count, Q5's `FILTER` aggregates and Q6's `date_trunc ... AT TIME ZONE` have no clean LINQ translation. Each statement is 003_DB's text, run as an ADO.NET command on the EF connection and transaction with every value an `NpgsqlParameter` (WI-004 DEC-024), and read by ordinal into small row records (`StatusCountRow` , `WorkloadRow` , `DashboardOrderRow` , `TopProductRow` , `DeliveryRow` , `TrendRow`). The statements are the SQL 003_DB reviewed, not a LINQ approximation of it.

Processing flow

Step	Description	Calls
1	<code>await using var tx = await db.Database.BeginTransactionAsync(IsolationLevel.RepeatableRead, ct)</code>	EF Core / Npgsql
2	<code>SET TRANSACTION READ ONLY</code> — the first statement in the transaction, before any query, as PostgreSQL requires. Any write in this transaction would now fail, which guards the dashboard's "changes no data" (REQ-039) at the database	<code>ExecuteSqlRawAsync</code> (constant text, no input)
3	Q1 status counts	<code>SqlQuery<StatusCountRow></code>
4	Q2 workload buckets with <code>Today</code> , <code>Week0</code>	<code>SqlQuery<WorkloadRow></code>
5	Q3a overdue (<code>due_date < Today</code>), Q3b due soon (<code>due_date BETWEEN Today AND SoonEnd</code>)	<code>SqlQuery<DashboardOrderRow></code> × 2
6	Q4 top products	<code>SqlQuery<TopProductRow></code>
7	Q5 delivery row with the five instants and the zone	<code>SqlQuery<DeliveryRow></code>
8	Q6 trend with <code>TrendStartUtc</code> , <code>EndUtc</code> and the zone	<code>SqlQuery<TrendRow></code>
9	<code>await tx.CommitAsync(ct)</code> (a read-only commit releases the snapshot) and return <code>DashboardRaw</code>	—

The statements run sequentially on one connection; a transaction cannot run them in parallel, and at 003_DB's volumes the total is a few milliseconds. The Npgsql command timeout stays at its 30 s default, as in 002_DD.

4. DashboardMapper.ToResponse

Field	Value
Description	Shape the raw rows into the API response
Return type	<code>DashboardResponse</code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	DashboardWindow	window	Bucket dates and window starts
2	DashboardRaw	raw	Reader output
3	DateTimeOffset	asOf	The single clock reading
4	string	timeZoneId	Echoed as <code>timeZone</code>

Return value

Type	Name	Description
DashboardResponse	response	See 003_DD-API

Processing overview: a pure function. Every "missing means zero" rule, and the one rounding rule the server owns, live here, so they are unit-tested without a database.

Processing flow

Step	Description	Calls
1	Status counts: look up each of the four statuses in Q1's rows, 0 when absent; <code>total</code> = their sum	—
2	Workload: build exactly 10 entries in display order — <code>overdue</code> (bucket -1); $8 \times \text{week}$ for $k = 0 \dots 7$ with <code>weekStart</code> = <code>Week0</code> + $7k$, except $k = 0$, where it is <code>Today</code> , and <code>weekEnd</code> = <code>Week0</code> + $7k + 6$; then <code>later</code> (bucket 8). Each takes Q2's count and quantity for its bucket, 0 when absent	—
3	Overdue / due soon: <code>total</code> = the first row's window count, or 0 when there is no row; <code>orders</code> = the rows mapped to <code>DashboardOrder</code>	—
4	Top products: map as returned	—
5	Completed this week / month: counts, <code>NULL</code> sums $\rightarrow 0$, with <code>from</code> = <code>Week0</code> / <code>MonthStart</code>	—
6	On time: { <code>onTimeCount</code> = x , <code>completedCount</code> = y , <code>windowStart</code> = <code>WindowStart</code> }	—
7	Lead time: <code>averageDays</code> = $y == 0 ? \text{null} : \text{Math.Round}(\text{avg}, 1, \text{MidpointRounding.AwayFromZero})$, <code>orderCount</code> = y	—
8	Trend: for each of the 12 <code>TrendWeeks</code> , the Q6 count whose <code>week_start</code> equals it, 0 when absent; <code>weekEnd</code> = <code>weekStart</code> + 6	—
9	<code>asOf</code> , <code>today</code> , <code>timeZone</code>	—

An integrity guard is asserted in the unit tests, not at run time: the workload counts sum to `draft + inProgress` , and Q3a's total equals the `overdue` bucket's count. Both follow from the single snapshot (DEC-015).

5. `IPlantClock` — additive members

Field	Value
Description	Three members the dashboard's calendar needs, added to the existing port without changing <code>Today</code> or <code>CurrentYear</code>
Return type	see below
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments / return values

Member	Signature	Behavior
<code>DateOf</code>	<code>DateOnly DateOf(DateTimeOffset utc)</code>	The plant-local date of an instant. <code>Today</code> becomes <code>DateOf(timeProvider.GetUtcNow())</code> , so the two can never disagree
<code>StartOfDayUtc</code>	<code>DateTimeOffset StartOfDayUtc(DateOnly date)</code>	The UTC instant at which that plant-local date begins: local midnight converted with <code>TimeZoneInfo</code> . When a DST gap removes local midnight, the first valid local time of that day is used. <code>Asia/Tokyo</code> has no DST; the rule exists so a reconfigured plant zone stays correct
<code>TimeZoneId</code>	<code>string TimeZoneId { get; }</code>	The configured IANA id (<code>PlantOptions.TimeZone</code>), validated at startup. Passed to PostgreSQL as <code>@zone</code> : PostgreSQL and .NET both read IANA tzdata, so they agree on the zone's rules

Processing overview: `PlantClock` already resolves the `TimeZoneInfo` once. These members reuse it, so no other class converts time zones.

6. `SystemHealthService.CheckAsync`

Field	Value
Description	Report whether the database answers <code>SELECT 1</code> within 2 seconds
Return type	<code>Task<SystemHealthResponse></code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments

No	Type	Name	Description
1	CancellationToken	cancellationToken	The request's token

Return value

Type	Name	Description
SystemHealthResponse	health	{ Database = "ok" \ "unavailable", CheckedAt } — never an exception for a failed ping

Processing overview: the port `IDatabasePing.PingAsync(ct)` is implemented by `DatabasePing` as `db.Database.ExecuteSqlRawAsync("SELECT 1", ct)` (constant text, no input). The service links the request token with a 2-second `CancellationTokenSource`. The port exists so the integration tests can replace it with a failing or a slow implementation (TC-224) without stopping the shared database container.

Processing flow

Step	Description	Calls
1	Start span <code>System.Health</code>	<code>ProductionOrderTelemetry.Source</code>
2	<code>using var timeout = CancellationTokenSource.CreateLinkedTokenSource(ct); timeout.CancelAfter(2 s)</code>	—
3	<code>await ping.PingAsync(timeout.Token) → database = "ok"</code>	<code>IDatabasePing</code>
4	On <code>OperationCanceledException</code> caused by the timeout (not by <code>ct</code>), or on <code>DbException</code> / <code>NpgsqlException</code> / <code>InvalidOperationException</code> from the connection → <code>database = "unavailable"</code> ; log <code>Warning DatabasePingFailed</code> with the exception type only; span status <code>Error</code>	logger
5	If <code>ct</code> itself was cancelled (the client went away), rethrow — nothing to report	—
6	Counter <code>health_checks{database}</code> ; return <code>{ database, CheckedAt = timeProvider.GetUtcNow() }</code>	<code>telemetry</code> , <code>TimeProvider</code>

7. Session renewal rule (`OnCheckSlidingExpiration`)

Field	Value
Description	Requests to <code>/api/system/health</code> never renew the authentication cookie (DEC-019)
Return type	— (cookie options event)
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Processing overview: a cookie can be renewed from two places, so both are covered (WI-004 DEC-025). In `ConfigureApplicationCookie`, alongside the existing `OnRedirectToLogin / OnRedirectToAccessDenied` handlers: (1) `Events.OnCheckSlidingExpiration` sets `ShouldRenew = false` for the health path; (2) Identity's `Events.OnValidatePrincipal` (the security-stamp validator, which renews after its 30-minute interval) is wrapped — it runs unchanged, then `ShouldRenew = false` for the health path, so a revoked session is still rejected. Nothing else in the cookie configuration changes: expiry, sliding for every other path, `HttpOnly`, `SameSite=Lax`. The path is a constant shared with the controller's route, so the two cannot drift apart. Verified by TC-225 with the cookie handler's `TimeProvider` advanced past half the lifetime.

Observability

Extends the existing `ProductionManagementAI.ProductionOrders` source and meter, per `ai/rules/backend.md`.

Signal	Name	Type	Attributes / buckets
Span	<code>ProductionOrder.Dashboard</code>	Activity	<code>dashboard.today</code> (plant date), <code>dashboard.active_orders</code> , <code>dashboard.window_completed</code> ; status <code>Error</code> on failure. The Npgsql instrumentation already in <code>Program.cs</code> adds a child span per statement, which shows the seven queries' individual cost
Metric	<code>pmai.production_orders.dashboard_loaded</code>	Counter	<code>outcome = success error</code>
Span	<code>System.Health</code>	Activity	status <code>Error</code> when the ping fails
Metric	<code>pmai.system.health_checks</code>	Counter	<code>database = ok unavailable</code> — an outage shows as the <code>unavailable</code> share rising, with volume bounded by open dashboards ÷ 30 s

Logging: `Error` with the exception on failure (`DashboardSnapshotFailed`); `Warning` `DatabasePingFailed` with the exception type only (never the message, which can contain host or credential fragments from a connection error). Nothing on success — the counter and span cover it, and a dashboard load is not an auditable event. No figure or order data is logged.

Unresolved decisions

None. The behavior follows 003_BD D-01–D-09, 003_DB Q1–Q6, and DEC-008, DEC-009, DEC-011 and DEC-015. One implementation note is recorded rather than decided: if EF Core's `SqlQuery<T>` cannot map a composite row cleanly (for example, the nested product in `DashboardOrderRow`), the row type stays flat (`ProductId`, `Sku`, `Name`) and the mapper nests it. That changes no contract.